

modified:: 2022-12-05, 1439

up:: [Army Int Photobooth](#)

tags::

NS Photobooth Guide

Overview

The NS photobooth is made of two main parts: the frontend GUI and the backend server.

The backend server is distributed via a Docker image. Docker is a method for app distribution standardized in the cloud computing industry.

The frontend GUI is a web app. NodeJS is a Javascript runtime needed to run the development server to host the web app, although it is possible to use other web servers instead of NodeJS. Yarn, an alternative NodeJS package manager, is also needed to take advantage of the Yarn Zero-Installs feature enabled in the web app source code which is useful for air-gapped environments.

Table of Contents

- [Prerequisites](#)
- [Docker Container Setup \(First-time\)](#)
- [Starting the Photobooth frontend webserver](#)
- [Photobooth frontend GUI usage](#)
- [Photobooth backend usage](#)
- [Troubleshooting](#)

Prerequisites

Minimum Specs:

- 1000 series (Pascal) or greater Nvidia GPU with at least 4GB of VRAM
 - See <https://docs.nvidia.com/cuda/wsl-user-guide/index.html#wsl2-system-requirements>
- Decent CPU
- at least 8GB of RAM

i.e. most Nvidia gaming laptops should be fine.

Docker Desktop with GPU support & NodeJS runtime must be setup beforehand:

- [Docker Desktop Install Guide](#)
- [NodeJS Install Guide](#)

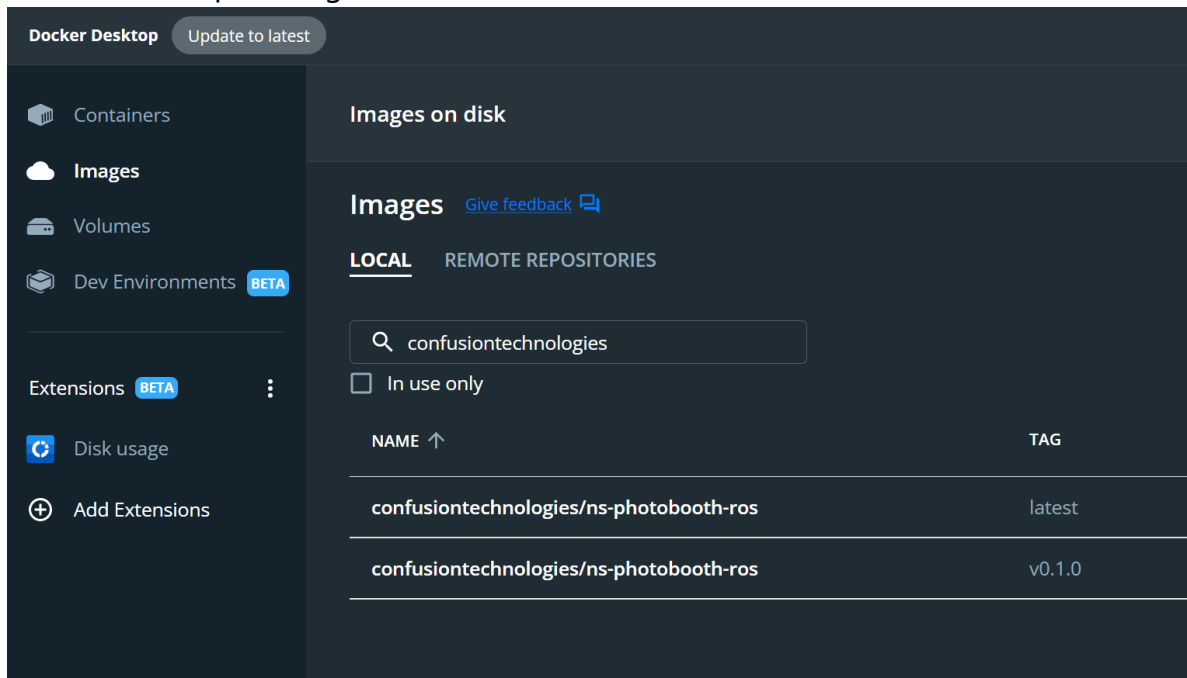
This guide assumes you already have the following files:

- `ns-photobooth-ros-vX.X.X.tar.xz`: Docker image tarball containing Photobooth backend.
- `ns-photobooth-main.zip`: Photobooth frontend UI source code.
- `props.json` (and prop images): JSON file mapping the name of the prop to the image to look for.
- `yolov5n6.onnx` and `vipnas_res50.onnx`: The human detection model and pose recognition model respectively.

Docker Container Setup (First-time)

Load the `ns-photobooth-ros` image into the local Docker repository and create a container instance from the image. The container is similar to a virtual machine and is reusable.

1. Run `docker load -i "path_to_image"`: Load the Docker image into the local Docker repository.
 - `path_to_image` should be the path to where `ns-photobooth-ros-vX.X.X.tar.xz` was downloaded onto the computer (i.e. `C:\Users\user\Downloads\ns-photobooth-ros-vX.X.X.tar.xz`).
 - The terminal will show multiple loading bars followed by the names of the loaded images after 2-3 mins.
 - `ns-photobooth-ros-vX.X.X.tar.xz` can be deleted/moved after this.
 - Skip this step if `confusionechnologies/ns-photobooth-ros` is already present in Docker Desktop > Images.



2. Create two folders anywhere (i.e. Documents > ns-photobooth) named `data` and `models`. Take note of their absolute locations as `path_to_data` and `path_to_models`.
 - Put `props.json` and the prop images into `data`.
 - Put `yolov5n6.onnx` and `vipnas_res50.onnx` into `models`.

3. Run `docker create -ti --name example_container --gpus all -p 9090:9090 -p 6080:6080 -v "path_to_data":/data -v "path_to_models":/models confusioantechnologies/ns-photobooth-ros`: Creates a container instance from the `ns-photobooth-ros` image. Options below have to be specified before the image name:
- `--name`: Give container a name to identify it with.
 - `--gpus all`: Give container full access to all GPUs.
 - `-p Y:X`: Expose port X inside the container as port Y on the host computer.
 - Port 9090 is used by the Photobooth API.
 - Port 6080 is used to host the VNC website for configuring the Photobooth backend.
 - `-v X:Y`: Symlink folder X on the host container to folder Y inside the container. Absolute path needed for both folders. Using quotes might be necessary if the path contains spaces.
 - `-ti`: `-t` allocates a TTY and `-i` makes STDIN interactive. Combined they allow opening an interactive shell in a Docker container.
 - Used in the `docker create` command as a workaround to keep the container running instead of shutting down immediately.
 - `docker create` documentation:
<https://docs.docker.com/engine/reference/commandline/create/%3E>

Starting the Photobooth frontend webserver

1. (First-time) Unzip `ns-photobooth-main.zip` to desired location (such as the Documents folder). Open a terminal inside the newly unzipped `ns-photobooth` folder and run `yarn`

(this "unpacks" the source code).

```
> yarn
> YN0000: Resolution step
> YN0002: @breq/roslib@npm:0.1.4 doesn't provide ws (p860fc
> YN0002: @breq/roslib@npm:0.1.4 doesn't provide ws (p3d57b
> YN0002: @preact/preset-vite@https://github.com/Interpause
cfeca800b5ecb48d3c32638e82a88 [8f4b9] doesn't provide preact
> YN0002: photobooth@workspace:.. doesn't provide react (p38
> YN0002: photobooth@workspace:.. doesn't provide react (p96
> YN0002: photobooth@workspace:.. doesn't provide react (pf1
> YN0002: photobooth@workspace:.. doesn't provide react (p7a
timer
> YN0000: | Some peer dependencies are incorrectly met; run y
etails, where <hash> is the six-letter p-prefixed code
> YN0000: | Completed in 0s 308ms
> YN0000: Fetch step
> YN0000: Completed
> YN0000: Link step
> YN0000: ESM support for PnP uses the experimental loader
> YN0008: esbuild@npm:0.14.38 must be rebuilt because its d
> YN0000: Completed in 0s 378ms
> YN0000: Done with warnings in 0s 931ms
```

2. Run `yarn vite` inside the unzipped folder to start the development webserver. It should automatically open the Photobooth frontend in the default browser.
 - There is also a `run.bat` shortcut that does this.

```
> yarn vite

vite v2.9.6 dev server running at:

> Local: http://localhost:3000/
> Network: use `--host` to expose

ready in 303ms.
```

See [Photobooth backend usage](#) for how to start the backend server.

Photobooth frontend GUI usage

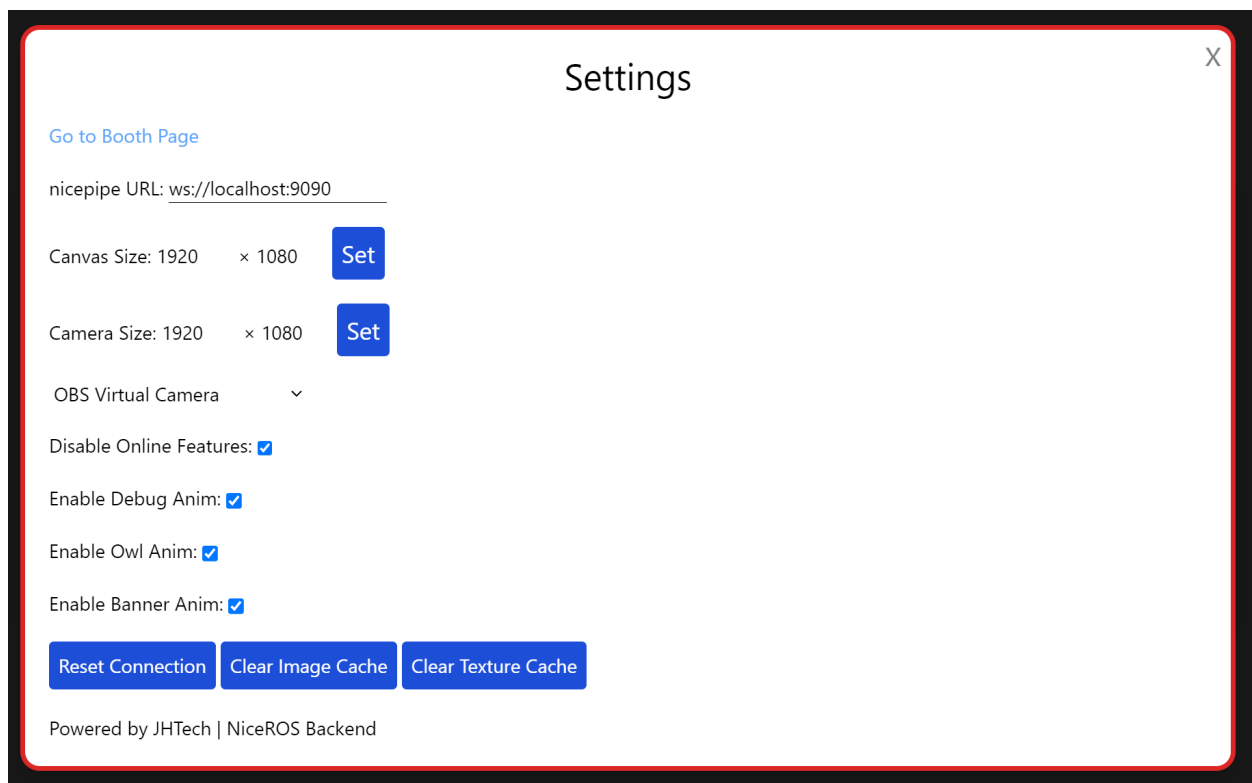
Keyboard Shortcuts

- `PageUp` (Presentation Clicker Button): Start photo countdown/Confirm photo
- `PageDown` (Presentation Clicker Button): Switch arrow to next person/Cancel photo
- `s`: Settings menu
- `r`: Reset connection
- `d`: Toggle debug visualization

Settings Menu

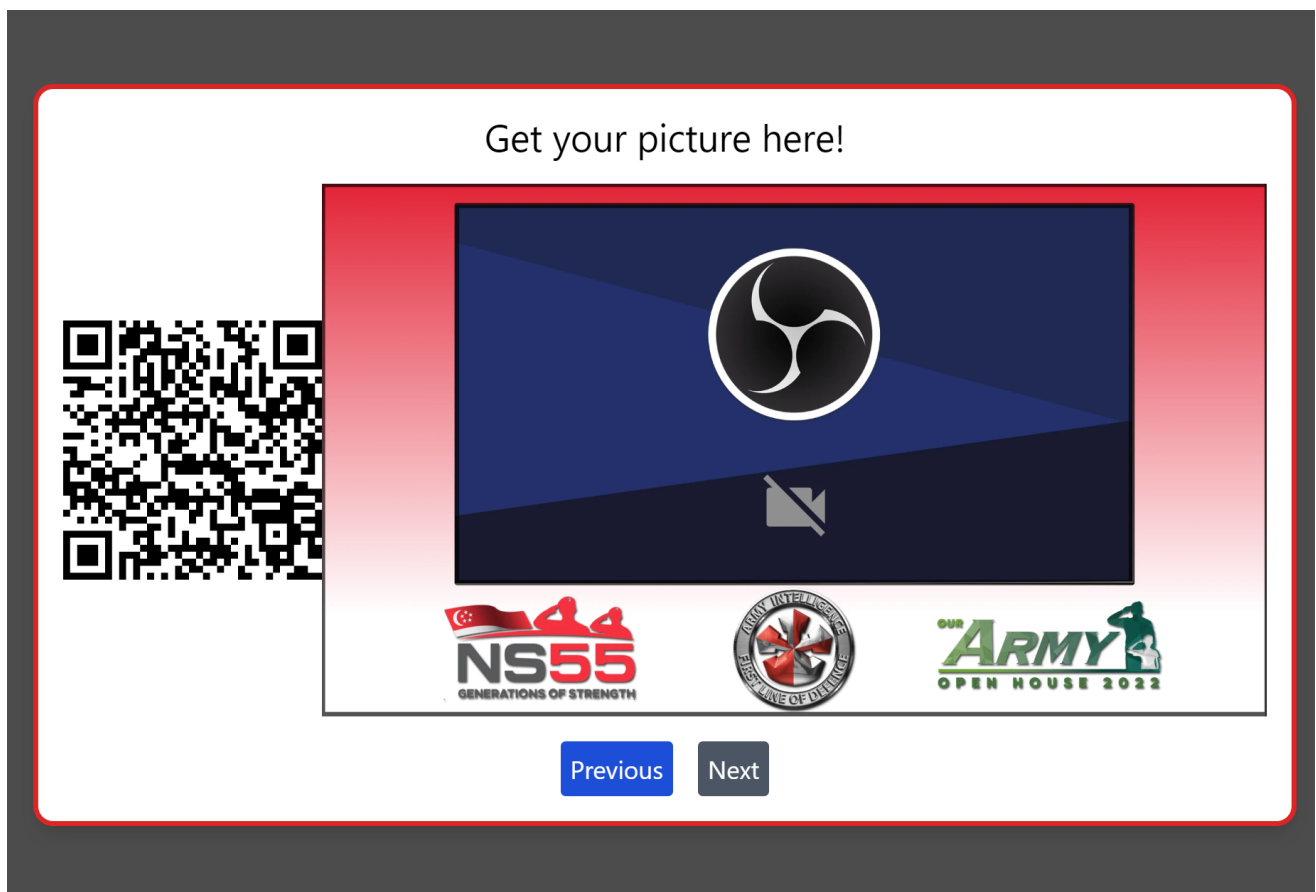
Either press **s** or find the invisible settings button in the top-left corner.

- Nicepipe URL: URL of the backend server. Most likely `ws://localhost:9090`.
- Canvas Size: Resolution for photos taken.
- Camera Size: Resolution requested from camera before being scaled to fit into the canvas.
- Select Camera...: Select the camera to use.
- Disable Online Features: (On By Default) Disables uploading the photos.
- Enable ... Anim: Enable/disable animations.
- Reset Connection: Reset connection to the backend.
- Clear Image Cache: Reset photo history on [QR Code Page](#).
- Clear Texture Cache: Reload textures used for animations.



QR Code Page

Go to <http://localhost:3000/qr>. Scanning the QR Code using your phone will open the [Photo Download Page](#).

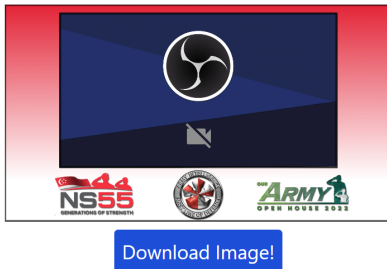


- Previous/Next: Browse previous photos taken.

Photo Download Page

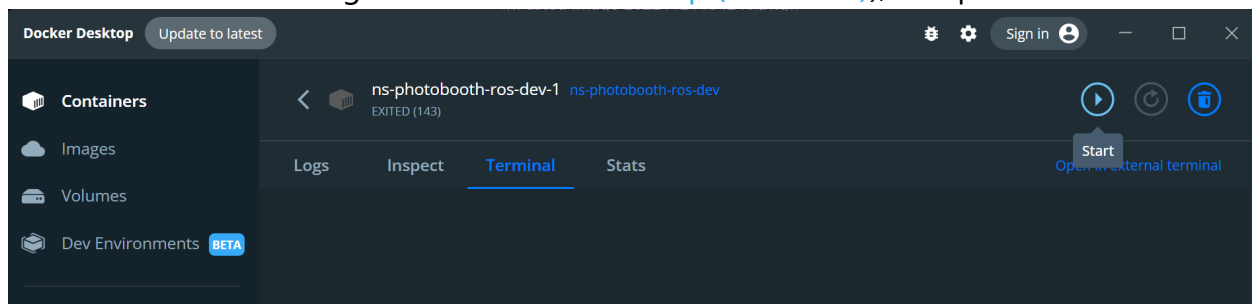
In the centre of the page is a preview of the image. If it is not present, the phone hasn't downloaded the image yet (bad connection?). Wait till the preview is present and the button changes from `loading...` to `Download Image!` before pressing the button. Refreshing the

webpage might help.

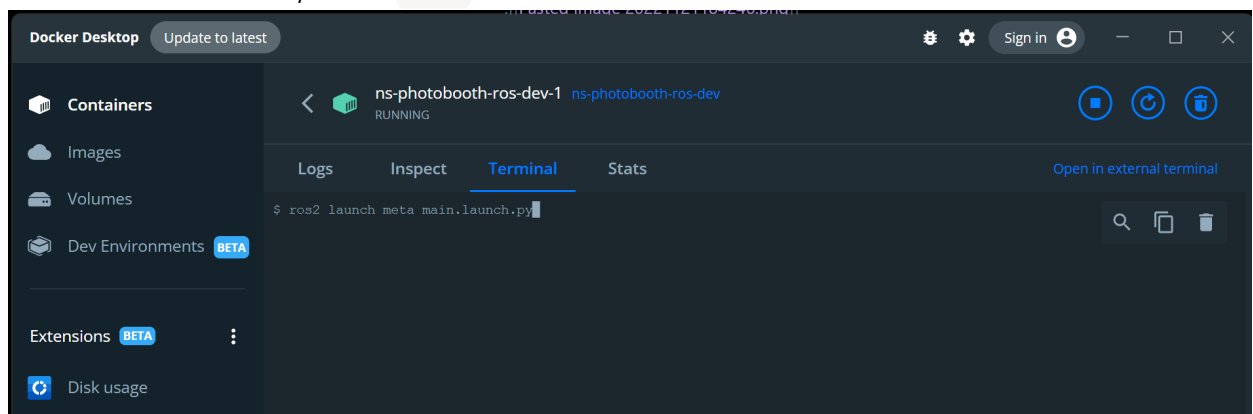


Photobooth backend usage

1. Go to Docker Desktop > Containers, click on the container created earlier (it should have been named during [Docker Container Setup \(First-time\)](#)), and press Start.

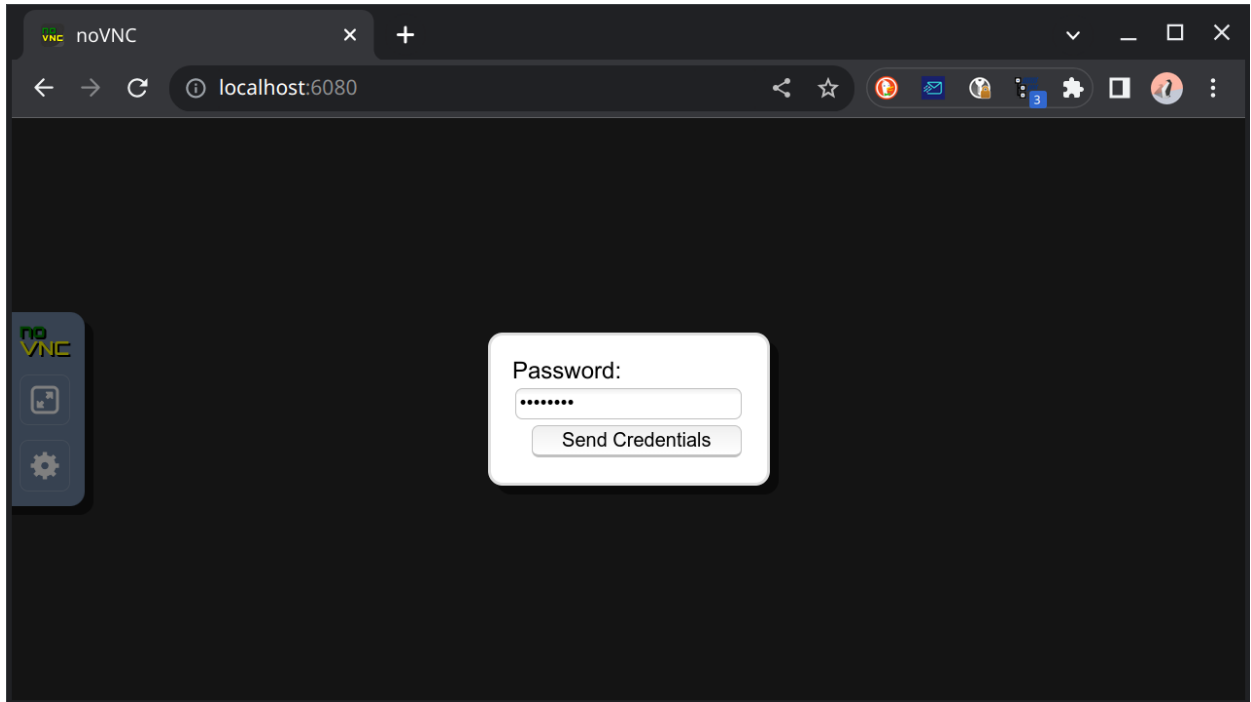


2. Run `ros2 launch meta main.launch.py` in the Terminal tab.
- If this doesn't work, enter `bash` first to switch to a Bash shell.

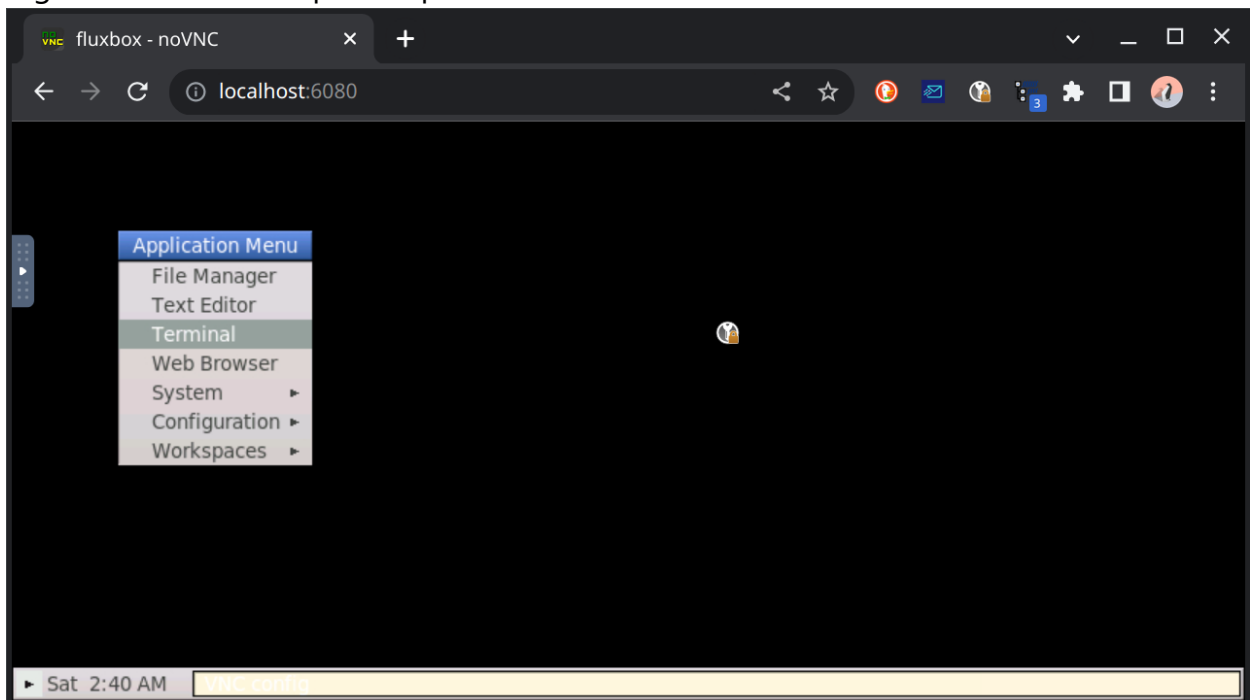


Prop calibration

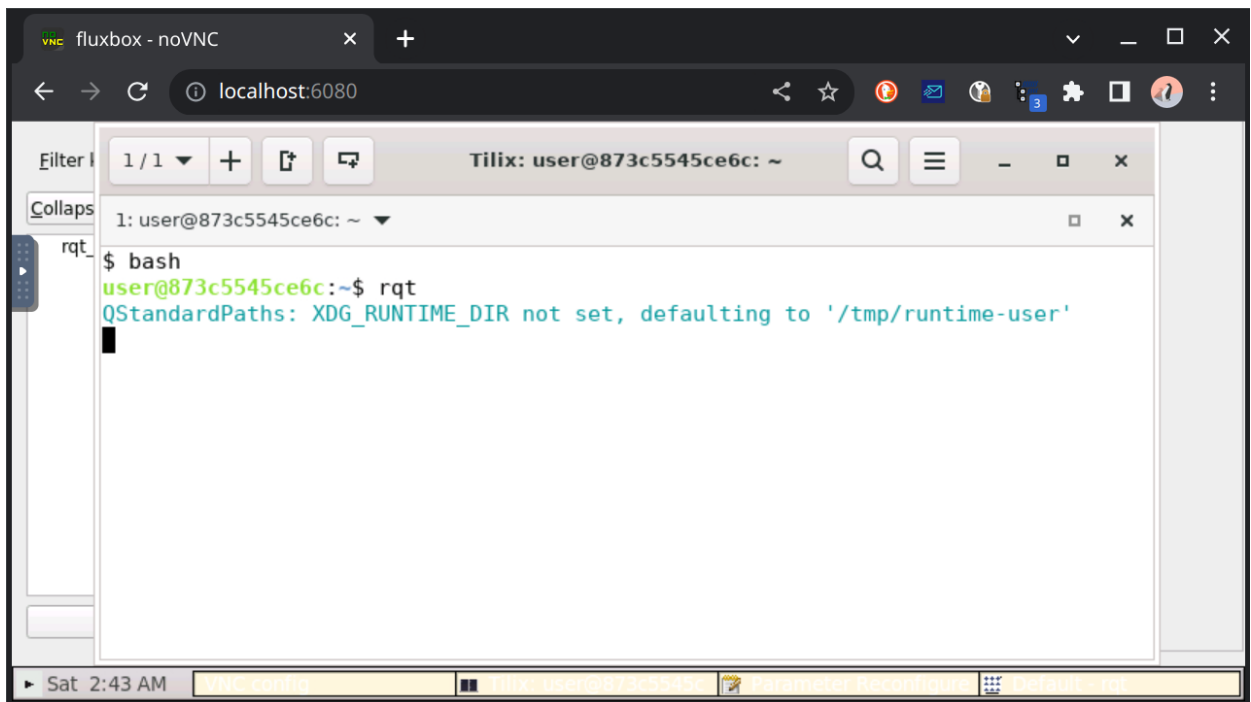
1. Open NoVNC (<http://localhost:6080>) and login (the password is "password").



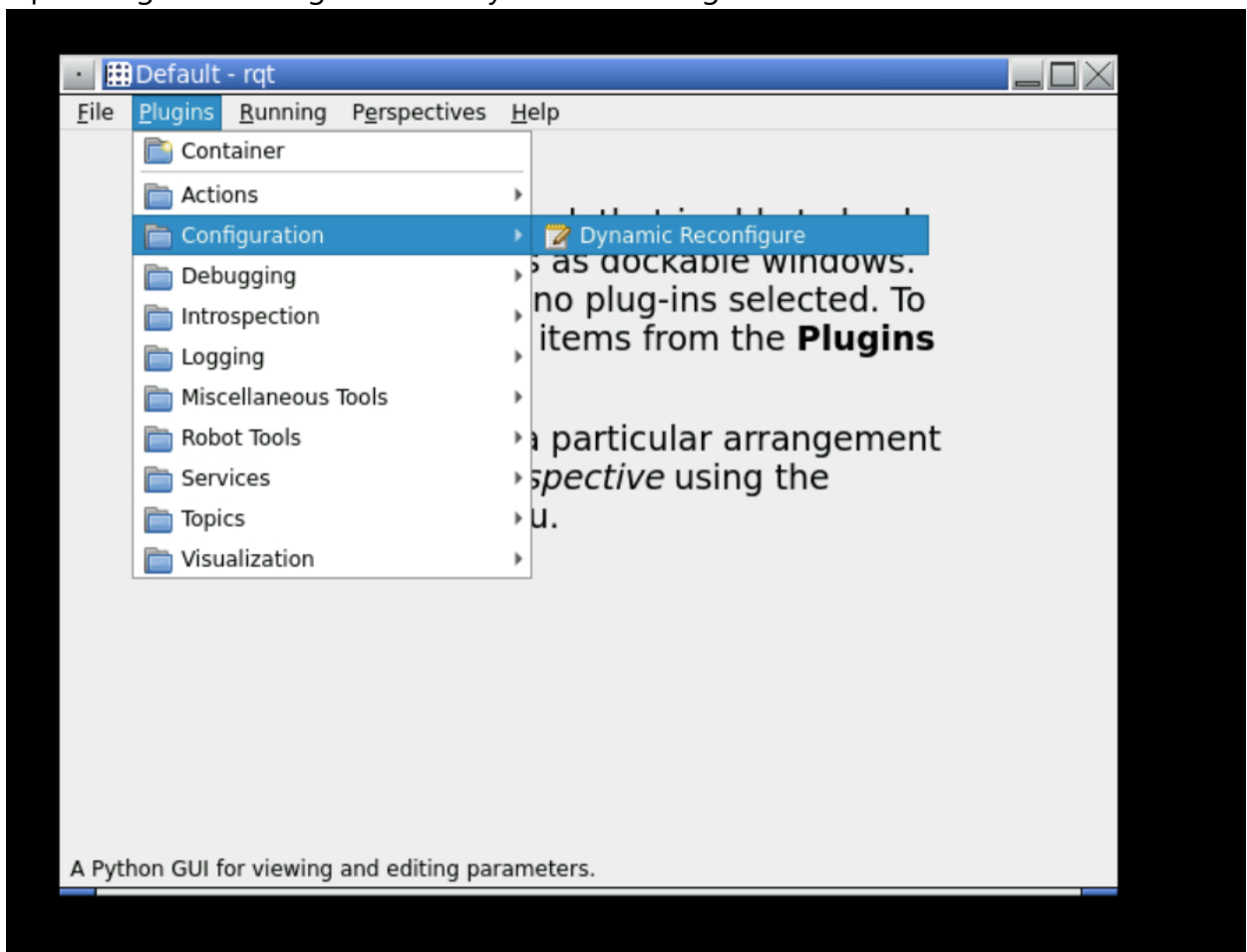
2. Right-click the desktop and open Terminal.



3. Enter `rqt` to launch rqt.
 - If this doesn't work, enter `bash` first to switch to a Bash shell.



4. Open Plugins > Configuration > Dynamic Reconfigure.



5. Press Refresh for all components to show up.

6. Click models > kpmatch.

Filter key:

▼ models

- kpmatch
- wholebody
- yolov5
- rosapi
- rosapi_params
- rosbridge_websocket
- rqt_gui_py_node_3...
- rtc
- rtc
- tracker

[/models/kpmatch](#)

use_sim_time	☐
rate	30.0
min_matches	40
use_flann	☒
use_cuda	☐
use_ocl	☐
scale_wh	640
normalize_coors	☒
ratio_thres	0.8
homography_method	8
inlier_thres	5.0
use_bg_subtraction	☒
database_path	/data/props.json
frames_in_topic	/rtc/rtc_receiver/frames_out
rect_out_topic	/rect_out
kp_out_topic	~/kp_out
debug	☐
query_detector.nfeatures	2000
query_detector.scaleFactor	1.1
query_detector.nlevels	12
query_detector.edgeThreshold	31
query_detector.firstLevel	0
query_detector.WTA_K	2
query_detector.scoreType	0
query_detector.patchSize	31
query_detector.fastThreshold	20
test_detector.nfeatures	20000

Brief explanation of how prop detection works:

1. Before runtime, a Query Detector extracts notable features of the prop images which are then stored.
2. During runtime, background subtraction is performed on a incoming live video frame.
3. A Test Detector extracts notable features of the video frame.
4. FLANN matcher matches similar looking features between the video frame and the prop images.
5. For each pair of matched features, the 2nd best pair is also found.
6. Lowe's Ratio Test filters out false positive matches by comparing the 1st and 2nd best pairs.
7. Homography filters the remaining matches as inliers and outliers and uses the inlier matches to calculate the detection box.

Options used for calibration:

- `min_matches`: Minimum number of matches to consider a prop as detected and attempt to calculate the detection box.
- `scale_wh`: Size to scale prop image to before extracting features.
- `ratio_thres`: Threshold for Lowe's Ratio Test to filter out false positive matched features. Filters out a match if similarity of the match is greater than the product of `ratio_thres` and similarity of the second best match.

- `homography_method`: Method used to find homography matrix for calculating the detection box from matched features.
 - See https://docs.opencv.org/4.6.0/d9/d0c/group_calib3d.html#ga4abc2ece9fab9398f2e560d53c8c9780 for supported methods. Methods are selected via enumeration.
 - See <https://amroamroamro.github.io/mexopencv/matlab/cv.findHomography.html> for an explanation of supported methods.
- `inlier_thres`: Threshold for `homography_method` to distinguish outlier matches from inliers when calculating the detection box.
- `test_detector` / `query_detector`: Query detector is used when extracting features from the prop images. Test detector is used when extracting features from the live feed.
 - Leave `test_detector` as it is as only one of the two detectors need to be multi-scale for the feature matching to work.
 - See https://docs.opencv.org/4.6.0/db/d95/classcv_1_1ORB.html#aeff0cbe668659b7ca14bb85ff1c4073b for documentation of the detector used.
 - `nfeatures`: Maximum number of features to select; Note it won't always use the maximum.
 - `nlevels` & `scaleFactor`: Number of times to zoom in and the zoom factor. For example, `nlevels = 4` & `scaleFactor = 1.2` would mean the detector would perform feature extraction at 4 levels, with the levels having a zoom factor of 1, 1.2, 1.44 and 1.728.
 - `firstLevel`: Which level to start from (counting from 0). Following the above example, `firstLevel = 2`, `nlevels = 4` & `scaleFactor = 1.2` would result in 1.44, 1.728, 2.074 and 2.488.
 - The rest of the config options are not important for calibration.

Troubleshooting

Enable the debug visualizations by pressing `d` to get a better idea of what failed.

No Cameras

Symptom: The list of cameras is empty.

There are two possible reasons:

1. Camera permissions are denied. Click the camera icon on the address bar to remedy this.
2. Microsoft Edge bug where information about cameras isn't given when the default camera is already in use or cannot load.

Camera Not Working

The camera may already be in use. On Windows, only one application can access the camera at a time. Ensure other camera applications are closed and no other tabs are using the camera.

To refresh the camera list (indirectly), change the values of "Camera Size" and press "Set". This can sometimes resolve cameras not being detected.

Props not being detected

Symptom: Green lines/boxes appearing and disappearing rapidly when debug visualization is on.

There are two possible reasons:

1. WebRTC is currently sending a low bitrate feed to the backend. Wait a bit longer for the connection to stabilize and WebRTC to increase the video quality (having the live feed be plain & still helps).
2. The photobooth backend needs calibration. See [Prop calibration](#).

Prop detection crashed

Symptom: Still able to detect people and track pose, but no green lines or boxes in debug visualization.

For unknown reasons, the prop detection can crash permanently after being left running for hours. Restart the container using Docker Desktop and follow [Photobooth backend usage](#) to start the backend again.

No detections at all

Symptom: No debug visualizations when debug visualizations turned on.

Press `Ctrl+Shift+I` to open the Developer Tools. Click on the Console tab and see what error is occurring. If it seems to be a connection issue, restart the container.

Typically, connection issues seem to happen on some computers with weird configurations. However, they do not randomly happen. Hence, any connection issue should be resolvable beforehand.

Error in Docker Desktop Terminal

CUDA ERROR

CUDA is a set of Nvidia drivers used to accelerate AI inference. CUDA errors are usually irrecoverable for a program (even if the program does not crash).

1. Restart the container.
2. If the error still persists, restart the computer.

FILENOTFOUNDERROR

Either `props.json` is missing, or one of the images referenced inside `props.json` is missing.

JSONDecodeError

`props.json` contains invalid JSON.